

Artikel Tugas UTS Pemrograman Web 2

by Akun cek

Submission date: 03-May-2026 11:24PM (UTC+0800)

Submission ID: 2950529727

File name: Artikel_Tugas_UTS_Pemrograman_Web_2.docx (28.32K)

Word count: 1131

Character count: 7390

Eksperimen SQL Injection pada Aplikasi Web dan Cara Mencegahnya

Pendahuluan

Perkembangan teknologi web yang semakin pesat membuat aplikasi berbasis web menjadi bagian penting dalam kehidupan sehari-hari. Mulai dari sistem akademik, e-commerce, hingga layanan publik, hampir semuanya kini bergantung pada aplikasi web. Namun, di balik kemudahan tersebut, terdapat ancaman keamanan yang tidak bisa diabaikan.

Salah satu ancaman yang paling umum adalah **SQL Injection**. Serangan ini memanfaatkan kelemahan dalam pengolahan input pengguna, sehingga penyerang dapat menyisipkan perintah SQL berbahaya ke dalam sistem.

Masalah ini sering terjadi karena developer kurang memperhatikan aspek validasi input atau menggunakan query database secara langsung tanpa pengamanan. Akibatnya, sistem menjadi rentan terhadap manipulasi data.

Dalam artikel ini, penulis akan melakukan eksperimen sederhana terhadap SQL Injection pada fitur login aplikasi web. Tujuannya adalah untuk memahami cara kerja serangan ini, melihat dampaknya secara langsung, serta mempelajari teknik pencegahannya.

Pembahasan Utama

1. Konsep Dasar SQL Injection

SQL Injection merupakan teknik eksploitasi keamanan yang dilakukan dengan cara menyisipkan perintah SQL ke dalam input pengguna. Jika aplikasi tidak melakukan validasi dengan baik, maka input tersebut akan diproses sebagai bagian dari query database.

Sebagai ilustrasi, berikut contoh query login sederhana:

```
$query = "SELECT * FROM user WHERE email='$email' AND password='$password'";
```

Query tersebut akan bekerja dengan normal jika input pengguna valid. Namun, jika input dimanipulasi, maka struktur query dapat berubah dan menghasilkan hasil yang tidak diinginkan.

2. Eksperimen SQL Injection

a. Lingkungan Pengujian

Eksperimen dilakukan pada aplikasi web sederhana berbasis PHP dengan framework CodeIgniter. Aplikasi memiliki fitur login yang terhubung dengan database MySQL.

Struktur tabel yang digunakan:

1. id
2. email
3. password

Form login menerima dua input:

1. Email
2. Password

b. Simulasi Serangan

Pada tahap ini, penulis mencoba memasukkan input berbahaya ke dalam kolom email:

' OR 1=1 –

Sedangkan password diisi sembarang.

Input ini dipilih karena merupakan teknik dasar SQL Injection yang sering digunakan.

c. Hasil Eksperimen

Setelah input dimasukkan dan form dikirim, sistem justru mengizinkan login meskipun password yang digunakan salah. Hal ini menunjukkan bahwa sistem memiliki celah keamanan. Jika dianalisis lebih lanjut, query yang dijalankan menjadi:

```
SELECT * FROM user WHERE email=" OR 1=1 -- ' AND password="
```

Penjelasan:

- **OR 1=1** selalu bernilai true (benar)
- -- digunakan untuk mengomentari sisa query

Akibatnya, kondisi WHERE selalu terpenuhi dan sistem menganggap login berhasil.

3. Analisis Kerentanan

Berdasarkan eksperimen yang dilakukan, dapat disimpulkan bahwa aplikasi memiliki kelemahan pada bagian input handling.

Beberapa faktor penyebabnya antara lain:

1. Tidak adanya validasi input
Input dari pengguna langsung diproses tanpa pemeriksaan.
2. Penggunaan query manual
Query ditulis secara langsung tanpa parameterisasi.
3. Tidak adanya proteksi database
Sistem tidak membatasi karakter atau pola input yang mencurigakan.

Dampak dari SQL Injection

Jika celah ini dimanfaatkan oleh pihak tidak bertanggung jawab, maka dampaknya bisa sangat serius:

- a. Akses ilegal ke akun pengguna
- b. Pencurian data sensitive
- c. Penghapusan atau perubahan data
- d. Pengambilalihan system

4. Teknik Pencegahan SQL Injection

a. Menggunakan Query Builder

Framework seperti CodeIgniter menyediakan Query Builder yang membantu mengamankan query.

```
$builder = $db->table('user');  
$builder->where('email', $email);  
$builder->where('password', $password);  
$query = $builder->get();
```

Query Builder secara otomatis melakukan escaping terhadap input.

b. Prepared Statement

Prepared statement memastikan bahwa input tidak dianggap sebagai bagian dari query.

```
$db->query(  
    "SELECT * FROM user WHERE email=? AND password=?",  
    [$email, $password]);
```

c. Validasi dan Sanitasi Input

Langkah ini meliputi:

1. Memastikan input tidak kosong
2. Menghapus karakter berbahaya
3. Menggunakan filter input

d. Enkripsi Password

Password tidak boleh disimpan dalam bentuk asli.

Gunakan:

```
password_hash()
```

```
password_verify()
```

Dengan cara ini, meskipun database bocor, password tetap aman.

e. Pembatasan Hak Akses Database

Gunakan prinsip **least privilege**, yaitu memberikan akses database sesuai kebutuhan saja.

Mengapa SQL Injection Masih Sering Terjadi ?

Meskipun SQL Injection sudah dikenal sejak lama, kenyataannya serangan ini masih sering ditemukan pada banyak aplikasi web. Hal ini biasanya bukan karena teknologi yang kurang, tetapi karena kesalahan dalam proses pengembangan. Banyak developer, terutama yang masih belajar, cenderung fokus pada fungsi aplikasi agar berjalan terlebih dahulu, tanpa memperhatikan aspek keamanan. Selain itu, penggunaan query secara langsung tanpa pengamanan masih sering dilakukan karena dianggap lebih mudah.

Faktor lain yang menyebabkan SQL Injection masih terjadi antara lain:

- a. Kurangnya pemahaman tentang keamanan web
- b. Tidak adanya validasi input pengguna
- c. Terburu-buru dalam pengembangan aplikasi
- d. Kurangnya pengujian keamanan sebelum aplikasi digunakan

Hal ini menunjukkan bahwa keamanan bukan hanya soal teknologi, tetapi juga kebiasaan dan kesadaran developer.

Studi Kasus Sederhana dalam Kehidupan Nyata

Untuk memahami dampak SQL Injection, bayangkan sebuah website toko online yang memiliki sistem login pelanggan.

Jika sistem login tersebut rentan terhadap SQL Injection, maka penyerang bisa:

1. Masuk ke akun orang lain tanpa password
2. Mengakses data pribadi pelanggan
3. Mengubah atau menghapus data pesanan

Dalam skala yang lebih besar, serangan ini bahkan bisa menyebabkan kebocoran data dalam jumlah besar, yang tentunya merugikan pengguna maupun pemilik sistem.

Pentingnya Keamanan dalam Pengembangan Web

Keamanan seharusnya tidak dianggap sebagai fitur tambahan, melainkan bagian utama dari pengembangan aplikasi. Sama seperti fitur login atau dashboard, keamanan juga harus dirancang sejak awal.

Beberapa prinsip dasar yang perlu diperhatikan oleh developer:

- a. Jangan pernah percaya input dari pengguna
- b. Selalu lakukan validasi dan filtering
- c. Gunakan fitur keamanan dari framework
- d. Lakukan pengujian sebelum aplikasi digunakan

Dengan menerapkan prinsip ini, risiko serangan dapat dikurangi secara signifikan.

Tips Praktis untuk Developer Pemula

Bagi developer pemula, berikut beberapa langkah sederhana yang bisa langsung diterapkan:

1. Gunakan framework seperti CodeIgniter atau Laravel

Framework biasanya sudah menyediakan sistem keamanan bawaan.

2. Hindari query manual
Gunakan Query Builder atau ORM.
3. Selalu gunakan password hashing
Jangan simpan password dalam bentuk asli.
4. Uji aplikasi sendiri
5. Biasakan membaca dokumentasi
Dokumentasi resmi sering menjelaskan praktik terbaik.

Relevansi SQL Injection di Era Sekarang

Di era digital saat ini, data menjadi aset yang sangat berharga. Banyak perusahaan menyimpan data penting seperti informasi pengguna, transaksi, hingga data keuangan. Jika sistem tidak aman, maka risiko kebocoran data menjadi sangat tinggi. Oleh karena itu, pemahaman tentang serangan seperti SQL Injection menjadi sangat penting, bahkan bagi developer pemula. Selain itu, kesadaran akan keamanan juga menjadi nilai tambah bagi seorang programmer, karena menunjukkan profesionalisme dalam bekerja.

Kesimpulan

SQL Injection merupakan salah satu celah keamanan yang sangat berbahaya namun masih sering terjadi dalam pengembangan aplikasi web. Melalui eksperimen yang dilakukan, terbukti bahwa sistem login yang tidak aman dapat dengan mudah ditembus hanya dengan manipulasi input sederhana. Hal ini menunjukkan pentingnya penerapan praktik keamanan dalam pengembangan aplikasi. Penggunaan query builder, prepared statement, serta validasi input merupakan langkah penting untuk menghindari serangan ini. Sebagai developer, memahami konsep keamanan bukan lagi pilihan, melainkan kebutuhan agar aplikasi yang dibangun dapat melindungi data pengguna dengan baik.

Referensi :

Rahardjo, Budi. 2019. *Keamanan Sistem Informasi*. Bandung: Informatika.

Kadir, Abdul. 2014. *Pengenalan Sistem Informasi Edisi Revisi*. Yogyakarta: Andi.

Enterprise Jubilee. 2018. *Keamanan Web dan Database*. Jakarta: Elex Media Komputindo.

Wahana Komputer. 2016. *Membangun Aplikasi Web dengan PHP dan MySQL*. Yogyakarta: Andi.

Dicoding Indonesia. 2023. *Belajar Dasar Keamanan Aplikasi Web*. Tersedia pada: <https://www.dicoding.com> (diakses 2 Mei 2026).

Artikel Tugas UTS Pemrograman Web 2

ORIGINALITY REPORT

5%

SIMILARITY INDEX

5%

INTERNET SOURCES

2%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

repository.unsoed.ac.id

Internet Source

1%

2

www.infosecinstitute.com

Internet Source

1%

3

kuyhaame.net

Internet Source

1%

4

www.ichi-hikaru.blogspot.com

Internet Source

1%

5

journal.student.uny.ac.id

Internet Source

1%

6

unimuda.e-journal.id

Internet Source

1%

Exclude quotes On

Exclude matches < 1%

Exclude bibliography Off